

Clases del Proyecto — Torneo Colegial

Clase: Usuario

Atributos (Modelo de Base de Datos y Esquemas Pydantic)

id: Entero autoincremental (Primary Key).

nombre_completo: String (Obligatorio).

email: String (Único, obligatorio, usado como identificador para el login).

password_hash: String (Obligatorio, almacena la contraseña encriptada, nunca en texto plano).

rol: Enum ("Administrador", "Entrenador", "Arbitro").

estado_activo: Boolean (Por defecto True. Permite suspender el acceso de una cuenta sin borrar sus registros históricos asociados).

fecha_creacion: DateTime (Generado automáticamente al insertar el registro, útil para auditoría).

Métodos (Repositorio - Operaciones SQL)

crear(usuario_in: UsuarioCreate) -> UsuarioModel: Inserta el nuevo registro (db.add, db.commit) y retorna la instancia generada.

obtener_por_email(email: str) -> UsuarioModel | None: Ejecuta un SELECT buscando coincidencia exacta; es el método principal para el login.

obtener_por_id(id_usuario: int) -> UsuarioModel | None: Recupera el usuario por su ID para validar relaciones (ej. asegurar que el árbitro asignado a un partido realmente existe).

modificar_estado(id_usuario: int, activo: bool) -> UsuarioModel: Hace un UPDATE del campo estado_activo para banear o habilitar cuentas.

Métodos (Servicio - Reglas de Negocio y Seguridad)

registrar_usuario(datos_registro: UsuarioCreate): Intercepta la petición, verifica si el email ya existe, aplica el algoritmo de hashing a la contraseña plana y envía el objeto listo al repositorio.

autenticar_usuario(email: str, password_plana: str) -> str: Llama al repositorio para obtener al usuario. Compara la contraseña plana con el password_hash. Si coincide y la cuenta está activa, firma y retorna un token JWT de sesión.

verificar_permisos(token: str, roles_permitidos: list[str]) -> bool: Función clave para FastAPI (Depends). Descripta el token JWT enviado en las cabeceras HTTP de cada petición, extrae el rol y verifica si coincide con los roles exigidos por la ruta.

Clase: Administrador (Hereda de Usuario)

Como el Administrador hereda de Usuario, su modelo de base de datos asume automáticamente los datos de autenticación (email, password_hash, estado_activo). Su existencia operativa se centra en los métodos de alto nivel para gobernar el torneo.

Atributos (Mapeados en models.py y validados en schemas.py)

Heredados de Usuario: id, nombre_completo, email, password_hash, estado_activo, fecha_creacion.

rol: Enum (Estrictamente forzado a "Administrador" por defecto al crear esta instancia).

nivel_acceso: String (Opcional. Útil si a futuro quieres tener un "SuperAdmin" que borre torneos y un "Coordinador" que solo programe partidos).

telefono_contacto: String (Para que los entrenadores o árbitros tengan un canal directo de soporte ante emergencias en las canchas).

Métodos (Implementados en el Servicio del Administrador)

A diferencia del Usuario general, los métodos del Administrador interactúan con los repositorios de casi todas las demás entidades.

Gestión del Torneo y Fases:

configurar_torneo(datos_torneo: TorneoCreate) -> TorneoModel: Crea la instancia principal de la competición, definiendo fechas y cupos.

estructurar_fases(id_torneo: int, configuracion_fases: dict): Define cuántos grupos habrá, cuántos clasifican por grupo y cómo se armarán las llaves (Octavos, Cuartos).

generar_programacion_general(id_torneo: int): Algoritmo maestro que cruza los equipos, asigna las Canchas disponibles y genera los objetos Partido.

Gestión de Staff y Participantes:

registrar_staff(datos: UsuarioCreate, rol_asignado: str): Único rol con permisos para crear cuentas de Entrenador y Árbitro en el sistema.

auditar_inscripcion_equipo(id_equipo: int, id_pago: int) -> bool: Revisa que el comprobante de pago sea válido, que el equipo tenga los jugadores mínimos y cambia su estado a "Habilitado".

asignar_arbitro_partido(id_partido: int, id_arbitro: int): Vincula a la autoridad de campo con el encuentro programado.

Control Disciplinario y Auditoría (Reglas de Excepción):

modificar_acta_cerrada(id_partido: int, nuevos_datos: dict, motivo_auditoria: str): Una vez que el árbitro cierra un partido, nadie puede cambiarlo excepto el Admin. Se usa para resolver apelaciones, alineaciones indebidas o victorias por W.O. (Walkover).

aplicar_sancion_disciplinaria(id_jugador: int, fechas_suspension: int): Bloquea a un jugador para que no pueda ser incluido en la Alineacion de los próximos partidos debido a agresiones o acumulación de tarjetas.

Clase: Entrenador (Hereda de Usuario)

Atributos (Mapeados en models.py y validados en schemas.py)

Heredados de Usuario: id, nombre_completo, email, password_hash, estado_activo, fecha_creacion.

rol: Enum (Estrictamente forzado a "Entrenador").

telefono_contacto: String (Obligatorio, necesario para la coordinación logística de los partidos).

colegio_id: Entero (Opcional/Foreign Key. Vincula al entrenador con la institución que representa, aunque la relación principal de pertenencia se da a través del Equipo).

Métodos (Implementados en el Servicio del Entrenador)

Sus métodos se enfocan exclusivamente en la gestión de su equipo, su plantilla y su participación en los partidos.

Gestión del Equipo y Plantilla:

registrar_equipo(datos_equipo: EquipoCreate) -> EquipoModel: Inscribe un nuevo equipo en un torneo específico (estado inicial "Preinscrito").

agregar_jugador_plantilla(id_equipo: int, datos_jugador: JugadorCreate) -> JugadorModel: Añade un jugador a la base de datos del equipo, validando que el equipo no haya superado el límite máximo de inscritos.

dar_de_baja_jugador(id_jugador: int): Desactiva a un jugador de la plantilla siempre y cuando el torneo lo permita (ej. antes de que cierre el mercado de pases).

Administrativo y Logístico:

cargar_comprobante_pago(id_equipo: int, datos_pago: PagoCreate): Sube la referencia de la transferencia o pago para que el Administrador valide la inscripción del equipo.

Competición (Día de Partido):

enviar_alineacion(id_partido: int, jugadores_convocados: List[int]): Selecciona de su plantilla general a los jugadores específicos (titulares y suplentes) que asistirán a un partido concreto. Crea los registros en la entidad Alineacion.

consultar_calendario(id_equipo: int) -> List[PartidoModel]: Retorna únicamente los partidos programados donde su equipo es local o visitante.

Clase: Árbitro (Hereda de Usuario)

Atributos (Mapeados en models.py y validados en schemas.py)

Heredados de Usuario: id, nombre_completo, email, password_hash, estado_activo, fecha_creacion.

rol: Enum (Estrictamente forzado a "Arbitro").

telefono_contacto: String (Obligatorio, vital para avisar sobre retrasos o problemas en la sede).

numero_colegiado: String (Opcional. Número de registro o credencial si pertenece a un colegio oficial de árbitros).

Métodos (Implementados en el Servicio del Árbitro)

Su alcance está estrictamente limitado a los partidos donde ha sido designado como autoridad. No puede ver ni alterar información de otros encuentros.

Consultas y Logística:

consultar_asignaciones(id_arbitro: int) -> List[PartidoModel]: Retorna la lista de partidos programados donde este árbitro ha sido asignado por el Administrador, incluyendo fecha, hora y la Cancha.

Ejecución del Partido (Día del evento):

iniciar_partido(id_partido: int): Cambia el estado del partido de "Programado" a "En_Juego".

registrar_evento(id_partido: int, datos_evento: EventoPartidoCreate) -> EventoPartidoModel: Añade una incidencia en tiempo real (Gol, Tarjeta Amarilla, Tarjeta Roja, Sustitución). Debe validar que el jugador involucrado esté en la Alineacion oficial de ese partido.

registrar_marcador(id_partido: int, goles_local: int, goles_visitante: int) -> MarcadorModel: Guarda el resultado numérico del encuentro.

Cierre y Validación:

cerrar_acta_oficial(id_partido: int): Acción crítica. Cambia el estado del partido a "Finalizado" y bloquea cualquier edición futura por parte del árbitro. Este método debe disparar un evento (o llamar a un servicio) que actualice automáticamente la TablaPosiciones y las estadísticas individuales (goleadores).

Clase: Colegio

Atributos (Mapeados en models.py y validados en schemas.py)

id: Entero autoincremental (Primary Key).

nombre: String (Obligatorio, único. Nombre de la institución).

direccion: String (Obligatorio, ubicación física de la institución).

representante_legal: String (Obligatorio. Persona responsable ante la organización del torneo).

telefono_contacto: String (Obligatorio, línea directa con el colegio).

email_institucional: String (Opcional, pero fuertemente recomendado para enviar notificaciones formales, resoluciones o facturas).

estado_activo: Boolean (Por defecto True. Útil si un colegio es sancionado o vetado de participar en futuros torneos, permitiendo bloquearlo sin borrar su historial de participaciones).

Métodos (Implementados en ColegioRepository y ColegioService)

Como organizador te digo que esta clase funciona como un "paraguas" administrativo. Su lógica no entra a la cancha, sino que organiza quién inscribe a quién.

Operaciones de Base de Datos (Repository):

crear_colegio(datos_colegio: ColegioCreate) -> ColegioModel: Inserta el registro de la institución en la base de datos (db.add, db.commit).

obtener_por_id(id_colegio: int) -> ColegioModel: Consulta y retorna la información completa del colegio.

obtener_todos(activos_solamente: bool = True) -> List[ColegioModel]: Lista para que el administrador vea todos los colegios que hacen parte de la red de torneos.

actualizar_detalle(id_colegio: int, datos: ColegioUpdate) -> ColegioModel: Modifica información de contacto o cambia al representante legal si el colegio lo solicita.

Reglas de Negocio y Relaciones (Servicio):

obtener_equipos_inscritos(id_colegio: int) -> List[EquipoModel]: Devuelve todos los equipos asociados a esta institución (ej. puede tener un equipo Sub-15 y otro Sub-17 en el mismo o en distintos torneos).

vincular_entrenador(id_colegio: int, id_entrenador: int): Enlaza a un usuario con rol de "Entrenador" a esta institución, permitiendo que actúe en su nombre.

Clase: Equipo

Atributos (Mapeados en models.py y validados en schemas.py)

id: Entero autoincremental (Primary Key).

colegio_id: Entero (Foreign Key referenciando a Colegio).

entrenador_id: Entero (Foreign Key referenciando a Usuario con rol "Entrenador").

torneo_id: Entero (Foreign Key. Vital para saber en qué competición específica participa esta instancia del equipo).

nombre: String (Obligatorio, ej. "Selección Prejuvenil").

escudo_url: String (Opcional. Ruta estática o URL del logo para la interfaz pública).

estado_inscripcion: Enum (Valores: "Preinscrito", "Pago_Pendiente", "Habilitado", "Descalificado").

Métodos (Implementados en EquipoRepository y EquipoService)

En la vida real, el equipo es el núcleo que conecta lo financiero (el pago), lo humano (entrenador y jugadores) y lo deportivo (las estadísticas y partidos).

Operaciones de Base de Datos (Repository):

crear_equipo(datos_equipo: EquipoCreate) -> EquipoModel: Ejecuta el INSERT del equipo asociándolo a su colegio, entrenador y torneo.

obtener_por_id(id_equipo: int) -> EquipoModel: Retorna los detalles del equipo junto con sus relaciones.

obtener_por_torneo(id_torneo: int) -> List[EquipoModel]: Devuelve la lista de equipos inscritos en un torneo específico (útil para generar el fixture).

actualizar_estado(id_equipo: int, nuevo_estado: str) -> EquipoModel: Ejecuta el UPDATE cuando el equipo pasa a estar "Habilitado" tras pagar, o "Descalificado" por una sanción.

Reglas de Negocio y Logística (Servicio):

validar_minimo_jugadores(id_equipo: int) -> bool: Cuenta los registros activos en la tabla Jugador asociados a este equipo. Debe retornar True solo si alcanza el mínimo exigido (ej. 11 jugadores).

bloquear_plantilla(id_equipo: int): Operación de control. Una vez que el torneo inicia o cierra el mercado, este método impide que se agreguen o eliminen jugadores de la base de datos del equipo.

verificar_solvensia(id_equipo: int) -> bool: Consulta la entidad Pago asociada para garantizar que el dinero fue auditado y aprobado antes de permitirles jugar el primer partido.

obtener_estadisticas_actuales(id_equipo: int) -> FilaPosicionModel: Consulta la entidad FilaPosicion para devolverle al entrenador un resumen de sus puntos, goles y rendimiento actual en el torneo.

Clase: Jugador

Atributos (Mapeados en models.py y validados en schemas.py)

id: Entero autoincremental (Primary Key).

equipo_id: Entero (Foreign Key referenciando a Equipo).

nombre_completo: String (Obligatorio).

identificacion: String (Único, obligatorio. Documento de identidad para evitar suplantaciones).

fecha_nacimiento: Date (Obligatorio, vital para las categorías por edad).

numero_camiseta: Entero (Opcional, puede actualizarse antes del partido).

posicion: String (Opcional. Ej: "Portero", "Defensa").

estado_activo: Boolean (Por defecto True. Si se retira del equipo, pasa a False).

sancionado: Boolean (Por defecto False. Se activa si tiene fechas de suspensión vigentes).

Métodos (Implementados en JugadorRepository y JugadorService)

A nivel de sistema, el jugador es casi siempre un ente pasivo. Las acciones recaen sobre él (lo inscriben, lo sancionan, le anotan goles), por lo que sus métodos son principalmente de validación y consulta.

Operaciones de Base de Datos (Repository):

crear_jugador(datos_jugador: JugadorCreate) -> JugadorModel: Inserta el registro del jugador asociado a su equipo en la base de datos.

obtener_por_id(id_jugador: int) -> JugadorModel: Retorna los detalles completos del jugador.

obtener_por_identificacion(identificacion: str) -> JugadorModel | None: Útil para asegurar que el mismo documento no se inscriba dos veces en el mismo torneo.

obtener_por_equipo(id_equipo: int) -> List[JugadorModel]: Retorna toda la plantilla de un equipo.

actualizar_estado_sancion(id_jugador: int, estado: bool) -> JugadorModel: Modifica el campo sancionado en la base de datos.

Reglas de Negocio y Elegibilidad (Servicio):

validar_requisitos_edad(fecha_nacimiento: date, categoria_torneo: str) -> bool: Calcula la edad exacta del jugador y la contrasta con los límites de la categoría del torneo (ej. Sub-17) antes de permitir su registro.

verificar_elegibilidad(id_jugador: int) -> bool: Consulta si el jugador tiene estado_activo en True y sancionado en False. Este método es llamado obligatoriamente cuando el Entrenador intenta meterlo en una Alineacion.

transferir_equipo(id_jugador: int, nuevo_equipo_id: int): Lógica para cambiar al jugador de equipo si las reglas del torneo permiten transferencias a mitad de temporada.

Clase: Pago

Atributos (Mapeados en models.py y validados en schemas.py)

id: Entero autoincremental (Primary Key).

equipo_id: Entero (Foreign Key referenciando a Equipo. Estrictamente necesario para saber quién está pagando).

monto: Decimal o Float (Obligatorio. Cantidad de dinero abonada).

fecha: DateTime (Obligatorio. Fecha y hora en la que se reporta la transacción).

metodo_pago: String (Obligatorio. Ej: "Transferencia", "Efectivo", "Consignación").

referencia_transaccion: String (Opcional, pero muy recomendado. Es el número de comprobante emitido por el banco).

comprobante_url: String (Opcional. Ruta o URL donde se almacena la foto/PDF del recibo de pago subido por el entrenador).

estado: Enum (Valores: "Pendiente", "Aprobado", "Rechazado").

Métodos (Implementados en PagoRepository y PagoService)

El pago es una entidad transaccional pura. Su ciclo de vida es corto: el entrenador lo crea, el administrador lo aprueba o rechaza, y su estado impacta directamente en la habilitación del equipo.

Operaciones de Base de Datos (Repository):

crear_pago(datos_pago: PagoCreate) -> PagoModel: Inserta el registro del pago vinculado al equipo, por defecto con estado "Pendiente".

obtener_por_id(id_pago: int) -> PagoModel: Retorna los detalles de un pago específico.

obtener_por_equipo(id_equipo: int) -> List[PagoModel]: Lista todos los intentos de pago de un equipo (útil si un primer pago fue rechazado y tuvieron que hacer otro).

actualizar_estado(id_pago: int, nuevo_estado: str) -> PagoModel: Ejecuta el UPDATE en la base de datos para cambiar el estado del pago.

Reglas de Negocio y Flujo Financiero (Servicio):

registrar_comprobante(id_equipo: int, datos_pago: PagoCreate): Método llamado por el Entrenador. Valida que los datos estén completos y envía la orden al repositorio para crear el registro.

validar_pago(id_pago: int, aprobado: bool, observaciones: str = None): Método exclusivo del Administrador. Si aprueba, cambia el estado a "Aprobado" y dispara una actualización en la clase Equipo para cambiar su estado_inscripcion a "Habilitado". Si es rechazado, guarda la observación (ej. "Comprobante ilegible").

Clase: Torneo

Atributos (Mapeados en models.py y validados en schemas.py)

id: Entero autoincremental (Primary Key).

nombre: String (Obligatorio. Ej: "Copa Intercolegial 2026").

categoría: String (Obligatorio. Ej: "Sub-17", "Femenino Libre").

fecha_inicio: Date (Obligatorio. Marca el día inaugural).

fecha_fin: Date (Obligatorio. Proyección del día de la final).

estado: Enum (Valores: "Inscripciones", "En_Curso", "Finalizado", "Cancelado").

reglas: Text o JSON (Opcional. Almacena configuraciones específicas como límite de jugadores, puntos por victoria, o un enlace al reglamento oficial).

Métodos (Implementados en TorneoRepository y TorneoService)

El torneo es la entidad raíz. Su estado dicta qué acciones están permitidas en el resto del sistema (por ejemplo, no se pueden inscribir jugadores si el torneo está "En_Curso").

Operaciones de Base de Datos (Repository):

crear_torneo(datos_torneo: TorneoCreate) -> TorneoModel: Inserta la configuración inicial del torneo en la base de datos.

obtener_por_id(id_torneo: int) -> TorneoModel: Retorna los detalles del torneo.

obtener_todos(estado_filtro: str = None) -> List[TorneoModel]: Retorna la lista de competiciones. Si se pasa un filtro, permite mostrar a los Espectadores solo los torneos activos.

actualizar_estado(id_torneo: int, nuevo_estado: str) -> TorneoModel: Ejecuta el UPDATE en la base de datos para avanzar el ciclo de vida del campeonato.

Reglas de Negocio y Control de Competición (Servicio):

iniciar_torneo(id_torneo: int): Valida que haya un número mínimo de equipos con estado "Habilitado". Cambia el estado del torneo a "En_Curso" y emite una orden para bloquear las plantillas de todos los equipos participantes.

estructurar_fases(id_torneo: int, config_fases: dict): Evoluciona la lógica estática de generarFixture() y generarArbolLlaves(). En su lugar, crea instancias de la clase Fase (ej. Grupos, Cuartos de Final) según la configuración dada por el Administrador.

finalizar_torneo(id_torneo: int): Cierra el evento cambiando el estado a "Finalizado", impidiendo cualquier modificación futura en partidos, tablas o plantillas asociadas a este.

Clase: Fase

Atributos (Mapeados en models.py y validados en schemas.py)

id: Entero autoincremental (Primary Key).

torneo_id: Entero (Foreign Key referenciando a Torneo. Conecta esta etapa a la competición matriz).

nombre: String (Obligatorio. Ej: "Fase de Grupos", "Octavos de Final", "Cuadrangular Final").

tipo_formato: Enum (Valores: "Grupos", "Eliminatoria_Directa". Crucial porque define si esta fase usa una TablaPosiciones o si avanza por eliminación directa).

estado: Enum (Valores: "Programada", "En_Juego", "Finalizada").

Métodos (Implementados en FaseRepository y FaseService)

Esta clase es la solución al problema de tener una bolsa infinita de partidos. Divide el torneo en bloques lógicos manejables.

Operaciones de Base de Datos (Repository):

crear_fase(datos_fase: FaseCreate) -> FaseModel: Inserta el registro de la etapa asociada al torneo.

obtener_por_id(id_fase: int) -> FaseModel: Retorna los detalles de la fase.

obtener_por_torneo(id_torneo: int) -> List[FaseModel]: Devuelve todas las etapas de un campeonato para armar la navegación en el frontend.

actualizar_estado(id_fase: int, nuevo_estado: str) -> FaseModel: Ejecuta el UPDATE para iniciar o concluir esta etapa.

Reglas de Negocio y Estructura (Servicio):

generar_fixture(id_fase: int): Crea automáticamente los objetos Partido para esta fase específica cruzando a los equipos participantes según el tipo_formato.

determinar_clasificados(id_fase: int) -> List[int]: Un método analítico. Si la fase fue "Grupos", lee la TablaPosiciones para sacar los IDs de los punteros. Si fue "Eliminatoria_Directa", lee los ganadores de los partidos de esa fase.

cerrar_fase(id_fase: int): Valida que todos los partidos pertenecientes a esta fase tengan estado "Finalizado". Si es así, cambia el estado de la fase a "Finalizada" y habilita la creación de la siguiente etapa.

Clase: Cancha

Atributos (Mapeados en models.py y validados en schemas.py)

id: Entero autoincremental (Primary Key).

nombre: String (Obligatorio. Ej: "Cancha Principal", "Sintética Norte").

direccion_sede: String (Obligatorio. Ubicación física, complejo deportivo o sede donde se encuentra).

tipo_superficie: String (Opcional. Ej: "Sintética", "Grama Natural", "Cemento").

estado_activo: Boolean (Por defecto True. Si la cancha entra en remodelación, se daña o no se renueva el contrato de alquiler, se pasa a False).

Métodos (Implementados en CanchaRepository y CanchaService)

Esta clase transforma un simple texto (String) en una entidad física gestionable, resolviendo el problema crítico de la doble reserva de escenarios deportivos.

Operaciones de Base de Datos (Repository):

crear_cancha(datos_cancha: CanchaCreate) -> CanchaModel: Registra un nuevo escenario deportivo en el sistema.

obtener_por_id(id_cancha: int) -> CanchaModel: Retorna los detalles de la cancha.

obtener_todas(solo_activas: bool = True) -> List[CanchaModel]: Lista los escenarios disponibles para que el Administrador pueda programar partidos.

modificar_estado(id_cancha: int, activo: bool) -> CanchaModel: Ejecuta el UPDATE para inhabilitar temporal o permanentemente una cancha.

Reglas de Negocio y Logística (Servicio):

verificar_disponibilidad(id_cancha: int, fecha: date, hora_inicio: time, hora_fin: time) -> bool: Método logístico vital. Consulta la entidad Partido para asegurar que no exista ningún otro encuentro programado en ese mismo bloque horario para esa cancha específica. Retorna True si está libre.

obtener_agenda(id_cancha: int, fecha_consulta: date) -> List[PartidoModel]: Devuelve todos los partidos programados para un día específico en esa cancha. Muy útil para el personal de logística de la sede física.

Clase: Partido

Atributos (Mapeados en models.py y validados en schemas.py)

id: Entero autoincremental (Primary Key).

fase_id: Entero (Foreign Key referenciando a Fase. Ubica el partido en su etapa del torneo).

equipo_local_id: Entero (Foreign Key referenciando a Equipo).

equipo_visitante_id: Entero (Foreign Key referenciando a Equipo).

cancha_id: Entero (Foreign Key referenciando a Cancha. Reemplaza tu antiguo String cancha para evitar errores tipográficos y de reservas).

arbitro_id: Entero (Foreign Key referenciando a Usuario con rol "Arbitro". Opcional al crearlo, pero obligatorio antes de iniciar).

fecha_hora: DateTime (Obligatorio. Momento exacto del pitazo inicial).

estado: Enum (Valores: "Programado", "En_Juego", "Finalizado", "Suspendido", "W_O").

Métodos (Implementados en PartidoRepository y PartidoService)

El partido es el corazón operativo del sistema. Aquí convergen la logística (cancha y fecha), el talento humano (equipos y árbitro) y los resultados (goles e incidencias).

Operaciones de Base de Datos (Repository):

crear_partido(datos_partido: PartidoCreate) -> PartidoModel: Inserta el cruce en la base de datos con estado "Programado".

obtener_por_id(id_partido: int) -> PartidoModel: Retorna toda la información del partido, idealmente haciendo un join para traer los nombres de los equipos y la cancha.

listar_por_fase(id_fase: int) -> List[PartidoModel]: Consulta masiva para pintar el fixture o calendario de una fase específica.

actualizar_estado(id_partido: int, nuevo_estado: str) -> PartidoModel: Ejecuta el UPDATE para transicionar el partido a lo largo de su ciclo de vida.

Reglas de Negocio y Control de Juego (Servicio):

programar_encuentro(datos_partido: PartidoCreate): Antes de guardar, invoca al servicio de la clase Cancha (verificar_disponibilidad) para asegurar que no haya cruces horarios, y valida que los dos equipos pertenezcan a la misma fase.

reprogramar_encuentro(id_partido: int, nueva_fecha: datetime, nueva_cancha: int): Lógica para casos de fuerza mayor (lluvia, apagones). Libera el bloque horario original y reserva el nuevo tras validarlo.

validar_alineaciones(id_partido: int) -> bool: Comprueba que ambos equipos (a través de sus entrenadores) hayan enviado sus instancias de Alineacion antes del pitazo inicial. Si un equipo no lo hace, se puede decretar un Walkover (W.O.).

finalizar_encuentro(id_partido: int): Método orquestador gigantesco. Se llama cuando el árbitro cierra el acta. Cambia el estado a "Finalizado" y lanza las órdenes para actualizar la TablaPosiciones y sumar las estadísticas individuales de cada jugador.

Clase: Alineacion

Atributos (Mapeados en models.py y validados en schemas.py)

id: Entero autoincremental (Primary Key). Aunque semánticamente es una tabla intermedia, tener un ID propio facilita las relaciones en SQLAlchemy.

partido_id: Entero (Foreign Key referenciando a Partido).

equipo_id: Entero (Foreign Key referenciando a Equipo. Identifica a qué bando pertenece esta planilla).

jugador_id: Entero (Foreign Key referenciando a Jugador).

es_titular: Boolean (Obligatorio. True si arranca en el campo, False si va al banco de suplentes).

numero_camiseta: Entero (Opcional. Permite sobrescribir el número del jugador solo para este partido, algo muy común en torneos amateur donde se prestan las camisetas).

Métodos (Implementados en AlineacionRepository y AlineacionService)

Esta clase es la barrera de seguridad de tu torneo. Es la planilla que el entrenador entrega antes de que el árbitro pite. Si esta entidad no existe, no tienes cómo evitar que jueguen expulsados o personas que ni siquiera están matriculadas en el colegio.

Operaciones de Base de Datos (Repository):

guardar_lista_jugadores(datos_alineacion: List[AlineacionCreate]):

Ejecuta un INSERT masivo (bulk insert) en la base de datos para guardar a todos los titulares y suplentes de una sola vez.

obtener_por_partido_y_equipo(id_partido: int, id_equipo: int) ->

List[AlineacionModel]: Retorna la planilla completa de un equipo para un partido específico.

eliminar_planilla(id_partido: int, id_equipo: int): Permite borrar los registros si el entrenador decide rehacer la alineación (solo permitido si el partido aún está con estado "Programado").

Reglas de Negocio y Seguridad Deportiva (Servicio):

enviar_planilla_oficial(id_partido: int, id_equipo: int, lista_jugadores:

List[dict]): El método central. Lo invoca el Entrenador. Antes de llamar al repositorio para guardar, debe ejecutar una validación estricta de las reglas del torneo.

validar_reglas_plantilla(lista_jugadores: List[dict]) -> bool: Valida que haya exactamente 11 jugadores marcados como es_titular = True (o los que exija el deporte), que no se exceda el límite de suplentes, y que no haya IDs duplicados.

auditar_elegibilidad_jugadores(lista_jugadores: List[int]): Cruza los IDs enviados con el servicio de la clase Jugador (verificar_elegibilidad). Si un solo jugador de la lista está inactivo o tiene el atributo sancionado = True, la transacción se rechaza por completo y lanza una excepción HTTP 400 (Bad Request) advirtiendo sobre el jugador inhabilitado.

Clase: Marcador

Atributos (Mapeados en models.py y validados en schemas.py)

id: Entero autoincremental (Primary Key).

partido_id: Entero (Foreign Key referenciando a Partido. Relación estricta de 1 a 1).

goles_local: Entero (Por defecto 0).

goles_visitante: Entero (Por defecto 0).

goles_penales_local: Entero (Opcional. Usado solo si el partido exige desempate desde el punto penal).

goles_penales_visitante: Entero (Opcional).

es_resultado_final: Boolean (Por defecto False. Pasa a True cuando el árbitro cierra el acta del encuentro, bloqueando modificaciones).

Métodos (Implementados en MarcadorRepository y MarcadorService)

Aunque técnicamente podrías guardar los goles como dos columnas extra en la tabla Partido, separar el Marcador en su propia clase te da una ventaja arquitectónica enorme. Te permite manejar la lógica de desempates, tiempos extra o victorias administrativas (W.O.) sin contaminar la entidad logística del partido.

Operaciones de Base de Datos (Repository):

crear_marcador_inicial(id_partido: int) -> MarcadorModel: Se dispara automáticamente al crear o iniciar un partido, insertando un registro en blanco (0-0).

obtener_por_partido(id_partido: int) -> MarcadorModel: Retorna el score actual de un encuentro específico.

actualizar_goles(id_partido: int, goles_local: int, goles_visitante: int) -> MarcadorModel: Sobrescribe los valores numéricos del resultado en la base de datos durante el juego.

fijar_resultado_final(id_partido: int) -> MarcadorModel: Ejecuta el UPDATE pasando es_resultado_final a True.

Reglas de Negocio y Lógica Deportiva (Servicio):

registrar_anotacion(id_partido: int, id_equipo_anotador: int): Método dinámico. Consulta si el equipo que anotó es el local o el visitante e incrementa el contador correspondiente en +1. Lo ideal es que este método sea invocado internamente cada vez que se registre un "Gol" en la clase EventoPartido.

aplicar_walkover(id_partido: int, id_equipo_ganador: int): Método disciplinario. Ignora los eventos de la cancha y establece el marcador automáticamente en 3-0 (o el estándar que dicten las reglas de tu torneo) a favor del equipo presente.

validar_necesidad_desempate(id_partido: int) -> bool: Se ejecuta al terminar el tiempo reglamentario. Revisa si el partido pertenece a una Fase con formato "Eliminatoria_Directa". Si hay empate, retorna True para mantener el acta abierta y exigir que se ingresen los resultados de los penales.

Clase: EventoPartido

Atributos (Mapeados en models.py y validados en schemas.py)

id: Entero autoincremental (Primary Key).

partido_id: Entero (Foreign Key referenciando a Partido. Vincula el evento a la línea de tiempo del encuentro).

equipo_id: Entero (Foreign Key referenciando a Equipo. Aunque se puede inferir del jugador, guardarlo aquí agiliza enormemente las consultas de estadísticas globales).

jugador_id: Entero (Foreign Key referenciando a Jugador. El actor principal: quien hace el gol, recibe la tarjeta o entra al campo).

jugador_secundario_id: Entero (Opcional. Foreign Key referenciando a Jugador. Vital para guardar quién dio la asistencia en un gol, o quién es el jugador que sale durante una sustitución).

tipo_evento: Enum (Valores: "Gol", "Asistencia", "Tarjeta_Amarilla", "Tarjeta_Roja", "Sustitucion", "Autogol").

minuto_ocurrencia: Entero (Obligatorio. Minuto de juego exacto, ej: 45, 89, lo que permite reconstruir el "minuto a minuto" para el frontend).

comentarios: String (Opcional. Texto libre para justificaciones del árbitro, ej: "Conducta violenta contra un adversario").

Métodos (Implementados en EventoPartidoRepository y EventoPartidoService)

Esta clase es la base de datos viva del torneo. De aquí nacen los goleadores, las suspensiones y el relato del partido. Arquitectónicamente, es excelente porque separa el registro histórico de las entidades físicas.

Operaciones de Base de Datos (Repository):

crear_evento(datos_evento: EventoPartidoCreate) -> EventoPartidoModel: Inserta la incidencia en la base de datos.

obtener_por_partido(id_partido: int) -> List[EventoPartidoModel]: Devuelve todos los eventos de un encuentro ordenados ascendentemente por el minuto_ocurrencia.

obtener_por_jugador(id_jugador: int, id_torneo: int = None) -> List[EventoPartidoModel]: Consulta masiva para extraer todas las tarjetas o goles de un individuo.

eliminar_evento(id_evento: int): Ejecuta un DELETE si el árbitro cometió un error tipográfico al registrar una acción (solo permitido mientras el partido no esté "Finalizado").

Reglas de Negocio y Consecuencias Reactivas (Servicio):

registrar_incidencia(datos_evento: EventoPartidoCreate): Método orquestador usado por el Árbitro. Antes de permitir el guardado, debe consultar la clase Alineacion para garantizar que el jugador_id realmente está convocado y jugando en ese partido.

procesar_efecto_secundario(evento: EventoPartidoModel): Actúa como un disparador (trigger) a nivel de lógica de aplicación. Si el evento es "Gol", invoca silenciosamente a

MarcadorService.registrar_anotacion(). Si es "Tarjeta_Roja", notifica al sistema disciplinario.

validar_sustitucion(id_partido: int, id_jugador_entra: int, id_jugador_sale: int): Lógica estricta de validación. Verifica que el jugador que entra esté marcado como suplente en la Alineacion, que el que sale esté en cancha, y verifica que el equipo no haya superado su límite máximo de cambios permitidos por el reglamento.

Clase: TablaPosiciones

Atributos (Mapeados en models.py y validados en schemas.py)

id: Entero autoincremental (Primary Key).

fase_id: Entero (Foreign Key referenciando a Fase. Indica a qué etapa del torneo pertenece esta clasificación).

nombre_grupo: String (Obligatorio. Ej: "Grupo A", "Grupo B", "Tabla General" o "Cuadrangular C").

estado_cierre: Boolean (Por defecto False. Pasa a True cuando se han jugado todos los partidos del grupo y las posiciones son matemáticas y definitivas).

Métodos (Implementados en TablaPosicionesRepository y TablaPosicionesService)

Arquitectónicamente, esta clase funciona como un contenedor o matriz. En lugar de tener los puntos de los equipos sueltos por ahí, la TablaPosiciones agrupa lógicamente a un conjunto de equipos (ej. los 4 equipos del Grupo A) para aplicarles reglas de ordenamiento y desempate sin que interfieran con los equipos del Grupo B.

Operaciones de Base de Datos (Repositorio):

crear_tabla(datos_tabla: TablaPosicionesCreate) ->

TablaPosicionesModel: Inserta el registro del grupo o tabla en la base de datos, vinculándolo a su fase.

obtener_por_fase(id_fase: int) -> List[TablaPosicionesModel]: Retorna todos los contenedores (grupos) que conforman una fase específica.

obtener_por_id(id_tabla: int) -> TablaPosicionesModel: Devuelve los datos del grupo (ideal para luego hacer un join con las filas de los equipos).

actualizar_estado_cierre(id_tabla: int, cerrado: bool) ->

TablaPosicionesModel: Bloquea la tabla para impedir que se sigan sumando puntos.

Reglas de Negocio y Ordenamiento (Servicio):

inicializar_grupos(id_fase: int, distribucion_equipos: dict): Método de configuración invocado por el Administrador (a través de la clase Fase). Crea las tablas ("Grupo A", "Grupo B") y automáticamente matricula a los equipos asignados inicializando todos sus contadores de puntos y goles en cero (creando instancias de FilaPosicion).

calcular_clasificacion_actual(id_tabla: int) -> List[FilaPosicionModel]: El motor matemático del sistema. Recupera todas las filas pertenecientes a este grupo y las ordena descendientemente aplicando los criterios de desempate en cascada estándar de la FIFA: 1. Mayor número de puntos; 2. Mayor diferencia de goles; 3. Mayor número de goles a favor; 4. Resultado del enfrentamiento directo.

auditar_y_cerrar_grupo(id_tabla: int): Método de control. Valida si todos los partidos programados para los equipos de esta tabla ya tienen estado "Finalizado". Si es así, pasa estado_cierre a True y retorna los IDs de los equipos que ocupan los puestos de clasificación para que el sistema los promueva a la siguiente ronda.

Clase: FilaPosicion

Atributos (Mapeados en models.py y validados en schemas.py)

id: Entero autoincremental (Primary Key).

tabla_id: Entero (Foreign Key referenciando a TablaPosiciones. Conecta esta fila con su grupo o fase correspondiente).

equipo_id: Entero (Foreign Key referenciando a Equipo. Identifica a quién le pertenecen estas estadísticas).

partidos_jugados (PJ): Entero (Por defecto 0).

partidos_ganados (PG): Entero (Por defecto 0).

partidos_empatados (PE): Entero (Por defecto 0).

partidos_perdidos (PP): Entero (Por defecto 0).

goles_a_favor (GF): Entero (Por defecto 0).

goles_en_contra (GC): Entero (Por defecto 0).

diferencia_goles (DG): Entero (Por defecto 0. Campo calculado numéricamente: GF - GC).

puntos (PTS): Entero (Por defecto 0. Métrica principal para el ordenamiento).

Métodos (Implementados en FilaPosicionRepository y FilaPosicionService)

Esta clase representa la unidad atómica de rendimiento de un equipo dentro de una fase. Su existencia evita tener que calcular en vivo todo el historial del torneo cada vez que un usuario abre la app para ver la tabla; en su lugar, simplemente se leen y ordenan estos registros precalculados.

Operaciones de Base de Datos (Repository):

crear_fila_inicial(id_tabla: int, id_equipo: int) -> FilaPosicionModel: Inserta el registro en cero cuando se genera un grupo nuevo.

obtener_por_tabla_y_equipo(id_tabla: int, id_equipo: int) -> FilaPosicionModel: Retorna la fila exacta de un equipo para actualizarla tras un partido.

actualizar_metricas(id_fila: int, datos_actualizados: FilaPosicionUpdate) -> FilaPosicionModel: Sobrescribe los contadores (puntos, goles, partidos) en la base de datos tras un recálculo.

Reglas de Negocio y Matemáticas (Servicio):

procesar_resultado_partido(id_tabla: int, id_equipo: int, goles_propios: int, goles_rivales: int): El método de cálculo puro. Toma los valores actuales de la base de datos y les suma los del último partido. Si goles_propios > goles_rivales, suma 1 a PG y 3 a PTS (dependiendo del reglamento). Además, actualiza automáticamente la diferencia_goles.

revertir_resultado_partido(id_tabla: int, id_equipo: int, goles_propios: int, goles_rivales: int): Un método de contingencia vital. Si un partido finalizado cambia por una demanda administrativa (ej. W.O. por alineación indebida), este método resta los puntos y goles del resultado original antes de sumar los nuevos, garantizando que la tabla no se corrompa.

Relaciones entre Clases

1. Relaciones de Herencia (Generalización)

Administrador, Entrenador y Árbitro → Usuario

Tipo: Herencia directa.

Descripción: Las tres clases de roles heredan los atributos base de autenticación y perfil (id, nombre_completo, email, password_hash, estado_activo) de la clase matriz Usuario, diferenciándose únicamente por los permisos de acceso y métodos específicos de su rol.

2. Relaciones de Composición (Contenedor y Ciclo de Vida)

Torneo → Fase

Descripción: Un torneo se divide de forma estricta en múltiples etapas (Fase). Las fases no tienen sentido analítico ni deportivo fuera del contexto de su torneo matriz.

Fase → TablaPosiciones

Descripción: Dependiendo del formato de la fase (ej. Fase de Grupos), esta contiene una o varias tablas de posiciones o grupos (como "Grupo A", "Grupo B").

TablaPosiciones → FilaPosicion

Descripción: Composición estricta. Una tabla de posiciones actúa como contenedor exclusivo de las filas estadísticas (FilaPosicion) de los equipos que participan en ese grupo.

Partido → Marcador

Descripción: Relación 1 a 1 de alta dependencia. Todo partido generado cuenta con un único objeto marcador que evoluciona junto con él hasta el cierre del acta.

Partido → EventoPartido

Descripción: Los eventos (goles, tarjetas, sustituciones) pertenecen de forma exclusiva a la línea de tiempo de un partido específico. Si el partido se elimina, sus incidencias históricas pierden validez operativa.

Equipo → Jugador

Descripción: Pertenencia de plantilla. Los jugadores se agrupan bajo la administración de un equipo durante el torneo.

3. Asociaciones y Pertenencia Operativa (Foreign Keys)

Colegio → Equipo

Descripción: Relación institucional (1 a N). Un colegio respalda, inscribe y funge como paraguas administrativo para uno o varios equipos en diferentes categorías.

Usuario (Entrenador) → Equipo

Descripción: Relación de gestión. El entrenador es el responsable operativo y deportivo de guiar a su equipo y enviar sus planillas.

Torneo → Equipo

Descripción: Asociación de participación. Los equipos se inscriben a un torneo específico y son filtrados por su estado de solvencia financiera y de plantilla.

Cancha → Partido

Descripción: Asociación logística. Los partidos apuntan a un ID de cancha obligatorio, lo que permite al sistema auditar la disponibilidad horaria y evitar cruces de espacio físico.

Usuario (Árbitro) → Partido

Descripción: Asociación de autoridad. El árbitro es asignado por el administrador para ejercer control sobre uno o varios partidos específicos.

Equipo → Pago

Descripción: Asociación financiera (1 a N). Cada intento o reporte de pago realizado por la institución o su entrenador queda vinculado directamente al equipo para su posterior validación administrativa.

Partido → Alineacion

Descripción: Asociación intermedia. Para cada partido, tanto el equipo local como el visitante generan registros de alineación que vinculan a sus jugadores convocados al terreno de juego.